Data Models

XML for Contracts

Tarapong Sreenuch

8 February 2026

克明峻德，格物致知

Building On: Two Enforcement Stacks

Topic 5 gave us the relational enforcement stack — constraints, access control, transactions — where the **database engine rejects** bad data for you.

Topic 6 flipped that: MongoDB accepts everything by default — the **application decides** what is valid.

Both assumed one thing: **the team that writes the data also controls the system that reads it.**

What happens when that is not true?

E-Invoicing Portal: 38 Invoices Stuck

A company's billing system generates XML invoices and submits them to a **Peppol-based e-invoicing portal**. The portal validates every submission against the **UBL (Universal Business Language) XSD** before accepting it.

Monday: 340 invoices submitted. 38 rejected. The error log:

```
element 'cbc:TaxAmount' is not expected at this position;
expected 'cbc:LineExtensionAmount'
```

Every rejected invoice is well-formed XML — tags match, the tree is valid syntax. But the UBL standard requires elements in a specific order. The billing system emits them backwards.

E-Invoicing Portal: Second Batch Fails

The developer fixes the element order. Wednesday, another batch fails:

```
element 'cbc:PaymentMeansCode' must have a value
from the UNCL4461 code list
```

The code sends `"bank_transfer"` — a readable string. The UBL standard requires a **numeric code** from a published list. The developer didn't know the list existed.

Cost: 38 invoices stuck for two days. Trade partners not paid. Two distinct errors — one structural, one value-based — both invisible without reading the contract.

Legacy Healthcare: 20 Developer-Hours Lost

A hospital group migrates patient records from a legacy system. The export format is **HL7 CDA** (Clinical Document Architecture) — XML with a 4,200-line XSD.

```
Without method:

  4,200-line XSD
  ├── grep "Patient"
  │   → 47 matches
  ├── grep "ID"
  │   → 83 matches
  ├── scroll, guess
  └── 30–90 min/field

  20 hours total
```

```
With contract card:

  Element: PatientID
  Parent: ClinicalDoc
  Compositor: sequence
  Cardinality: (1,1)
  Type: xs:string
  Pattern: ^P-\d{4}$

  ⬆ 2 minutes/field
```

The problem is not the developer's ability. The problem is the **absence of a systematic method** for reading the contract.

Payment API: $12,400 Reconciliation Gap

A fintech company receives payment notifications from a partner API. The documentation says: amount (number), currency (ISO 4217 string), paymentDate, merchantId.

A malformed notification arrives:

```
{
  "amount": "150.00",
  "currency": "sgd",
  "payment_date": "2025-02-13",
  "merchant_id": "M-2001"
}
```

The API accepts it. No validation. Three errors go undetected.

Payment API — Three Silent Errors

Error	What went wrong	When it surfaces
`"amount": "150.00"`	String, not a number	Analytics pipeline crashes on aggregation
`"currency": "sgd"`	Lowercase — not ISO 4217 `"SGD"`	Reconciliation report filters on exact match, misses it
`"payment_date"`	snake_case — consumer expects `paymentDate`	Webhook silently drops the field

Cost: $12,400 reconciliation gap discovered **5 days later**. 8 hours to trace the cause.

Root cause: the API has documentation but **no machine-checkable contract**. Documentation is a suggestion. A schema is a defence.

Why XML? Why Not Just JSON?

The company does not always choose the format — **the standard does**.

Domain	Standard	Format
E-Invoicing	Peppol / UBL	XML
Financial Reporting	XBRL	XML
Healthcare Exchange	HL7 CDA	XML
Customs Declarations	WCO standards	XML

These are domains where a regulator or industry body **publishes the schema** and every participant must conform. The format is not a design choice.

Failure 3 shows the same principle applies to JSON — once we establish the XSD skills, the same discipline carries to **JSON Schema**.

The Pattern: Three Failures, One Root Cause

	Failure 1 — Portal	Failure 2 — Legacy	Failure 3 — API
Data format	XML	XML	JSON
Problem	Well-formed but invalid	Contract unreadable	No contract at all
Cost	38 invoices stuck, 2 days	20 dev-hours wasted	$12,400 gap, 8 hours
Root cause	No local validation	No systematic reading method	No machine-checkable schema

All three share one root cause: **no machine-checkable contract was enforced where data crossed from one party to another**.

> Therefore, the engineer must **validate data against a contract at the boundary** — not discover structural errors in production, days after the damage is done.

Four Capabilities

What would have prevented these failures?

1. **Distinguish well-formed from valid** — a document can parse correctly and still violate the contract *(Failure 1)*
2. **Read the contract systematically** — given an XSD, extract a contract card for any element *(Failure 2)*
3. **Validate locally, before submission** — collapse the feedback loop from days to seconds *(Failure 1)*
4. **Extract fields from validated documents** — XPath on validated data returns deterministic results *(Failure 2)*

Capability 4 depends on 3, which depends on 2, which depends on 1. The operational chain is three steps — **parse** → **validate** → **extract** — powered by four capabilities. Capability 2 (reading the contract) is not a per-document step; it is the skill that makes validation usable. You read the contract once; you validate many times.

Vocabulary

Term	Definition
Well-formed	Correct XML syntax — matched tags, single root, proper nesting
Valid	Conforms to a schema — elements, order, cardinality, types all match
Element	A node in the XML tree — has a tag name, can contain children, attributes, text
Attribute	A name–value pair attached to an element (metadata, not a child)
Text node	Character data inside an element — the actual content (e.g., `"Kopi Tech Pte Ltd"`)
Namespace	A URI that qualifies element names to avoid collisions across vocabularies
Compositor	Ordering rule for child elements — `xs:sequence`, `xs:choice`, `xs:all`

Predict: an invoice XML has properly matched tags and a single root, but `<TaxAmount>` appears before `<LineExtensionAmount>`. Is it well-formed? Is it valid?

Well-Formed ≠ Valid

> **Well-formed** (yes) — the syntax is correct. Tags match, single root, proper nesting. **Valid** (no) — the element order violates the `xs:sequence` in the XSD.

This is exactly Failure 1: the document parses fine, but its structure violates the contract. The parser says "yes"; the validator says "no."

Valid implies well-formed — you cannot check tree shape if you cannot parse the tree. But **well-formed does not imply valid** — a syntactically correct tree can have wrong elements, wrong order, or missing required nodes.

Level	What it checks	Tool
Well-formed	Syntax: matched tags, root, nesting	`xmllint file.xml`
Valid	Structure: names, ordering, cardinalities, types	`xmllint --schema schema.xsd file.xml`

The XML Tree: Invoice as a Tree

```
Invoice
├── InvoiceNumber:  "INV-2025-0042"
├── IssueDate:      "2025-02-13"
├── Supplier
│   ├── Name:       "Kopi Tech Pte Ltd"
│   └── Address:    "88 Amoy Street, Singapore 069907"
├── Customer
│   ├── Name:       "Marina Bay Trading Co."
│   └── Address:    "10 Collyer Quay, Singapore 049315"
├── InvoiceLines
│   ├── InvoiceLine
│   │   ├── ItemName:    "Cloud Hosting (Annual)"
│   │   ├── Quantity:    1
│   │   └── LineAmount:  12000.00  [@currencyID="SGD"]
│   ├── InvoiceLine
│   │   ├── ItemName:    "SSL Certificate"
│   │   ├── Quantity:    5
│   │   └── LineAmount:  250.00    [@currencyID="SGD"]
│   └── InvoiceLine
│       ├── ItemName:    "Technical Support (Hours)"
│       ├── Quantity:    20
│       └── LineAmount:  3000.00   [@currencyID="SGD"]
├── MonetaryTotal
│   ├── LineExtensionAmount:  15250.00  [@currencyID="SGD"]
│   ├── TaxAmount:            1372.50   [@currencyID="SGD"]
│   └── PayableAmount:        16622.50  [@currencyID="SGD"]
├── PaymentMeans
│   ├── PaymentMethodCode:  "30"
│   └── PaymentDueDate:     "2025-03-15"
└── Note:  "Payment within 30 days of invoice date."
```

The tree is **ordered** — sibling order is meaningful. Compare: relational rows have no inherent order.

The XML Markup: Invoice INV-2025-0042

The tree above is a conceptual aid. This is the actual XML that produces it:

```
<?xml version="1.0" encoding="UTF-8"?>
<Invoice>
  <InvoiceNumber>INV-2025-0042</InvoiceNumber>
  <IssueDate>2025-02-13</IssueDate>
  <Supplier>
    <Name>Kopi Tech Pte Ltd</Name>
    <Address>88 Amoy Street, Singapore 069907</Address>
  </Supplier>
  <Customer>
    <Name>Marina Bay Trading Co.</Name>
    <Address>10 Collyer Quay, Singapore 049315</Address>
  </Customer>
  <InvoiceLines>
    <InvoiceLine>
      <ItemName>Cloud Hosting (Annual)</ItemName>
      <Quantity>1</Quantity>
      <LineAmount currencyID="SGD">12000.00</LineAmount>
    </InvoiceLine>
    <InvoiceLine>
      <ItemName>SSL Certificate</ItemName>
      <Quantity>5</Quantity>
      <LineAmount currencyID="SGD">250.00</LineAmount>
    </InvoiceLine>
   ...
```

```
    ...
    <InvoiceLine>
      <ItemName>Technical Support (Hours)</ItemName>
      <Quantity>20</Quantity>
      <LineAmount currencyID="SGD">3000.00</LineAmount>
    </InvoiceLine>
  </InvoiceLines>
  <MonetaryTotal>
    <LineExtensionAmount
currencyID="SGD">15250.00</LineExtensionAmount>
    <TaxAmount currencyID="SGD">1372.50</TaxAmount>
    <PayableAmount currencyID="SGD">16622.50</PayableAmount>
  </MonetaryTotal>
  <PaymentMeans>
    <PaymentMethodCode>30</PaymentMethodCode>
    <PaymentDueDate>2025-03-15</PaymentDueDate>
  </PaymentMeans>
  <Note>Payment within 30 days of invoice date.</Note>
</Invoice>
```

Each element opens with `<Tag>` and closes with `</Tag>`. `currencyID="SGD"` is an attribute. Compare each line to its corresponding node in the tree.

DTD: The Older, Simpler Schema

DTD (Document Type Definition) defines element structure using compact, non-XML syntax:

```
<!ELEMENT Invoice (InvoiceNumber, IssueDate, Supplier, Customer,
                   InvoiceLines, MonetaryTotal, PaymentMeans*, Note?)>
<!ELEMENT InvoiceLines (InvoiceLine+)>
<!ELEMENT InvoiceLine (ItemName, Quantity, LineAmount)>
<!ATTLIST LineAmount currencyID CDATA #REQUIRED>
```

Cardinality symbols: `?` = 0 or 1, `*` = 0 or more, `+` = 1 or more, no suffix = exactly 1.

All content is text (`#PCDATA`) — **DTD cannot express data types**.

DTD vs XSD: Why Standards Use XSD

Capability	DTD	XSD
Data types (`integer`, `date`, `decimal`)	No — all content is `#PCDATA`	Yes
Value restrictions (enumerations, ranges, patterns)	Limited — attribute enumerations only	Yes
Namespace support	No	Yes
Schema language format	Non-XML (own syntax)	XML — the schema is itself XML
Reusable type definitions	No	Yes — named `complexType` / `simpleType`

A standard that can only say "this element contains text" is too weak to prevent the errors that caused real financial consequences in Failure 1.

XSD Compositors: Ordering Rules

XSD defines XML structure using XML itself. The first key construct: **compositors** control child element ordering.

Compositor	Rule	Example
`xs:sequence`	Children must appear in declaration order	Failure 1's element-order rejection
`xs:choice`	Exactly one of the listed children appears	`CreditNote` or `DebitNote`
`xs:all`	All listed children, any order	Relaxes ordering; less common

Failure 1's rejection — `TaxAmount` before `LineExtensionAmount` — is a compositor violation. The `xs:sequence` in the XSD defines the only acceptable order.

XSD Cardinality: (min, max) Returns

The second key construct: **cardinality** — the same (min, max) pairs from Topic 2, now applied to tree children.

XSD cardinality	(min, max)	Meaning	Topic 2 parallel
`minOccurs="1" maxOccurs="1"` (default)	(1,1)	Exactly one	FK + NOT NULL + UNIQUE
`minOccurs="0" maxOccurs="1"`	(0,1)	Optional	FK + UNIQUE (nullable)
`minOccurs="1" maxOccurs="unbounded"`	(1,N)	One or more	FK + NOT NULL
`minOccurs="0" maxOccurs="unbounded"`	(0,N)	Zero or more	FK (nullable)

Not a loose analogy — the **same concept** in a different notation. Topic 2's (min, max) constrained entity participation in relationships; XSD's `minOccurs`/`maxOccurs` constrains child elements in a parent. Failure 1's `<InvoiceLine>` with `minOccurs="1"` is Topic 2's "total participation" — **at least one must exist**.

The Complete Invoice XSD Contract

```
<!— Root element declaration —>
<xs:element name="Invoice">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="InvoiceNumber" type="xs:string"/>
      <xs:element name="IssueDate" type="xs:date"/>
      <xs:element name="Supplier" type="PartyType"/>
      <xs:element name="Customer" type="PartyType"/>
      <xs:element name="InvoiceLines"
type="InvoiceLinesType"/>
      <xs:element name="MonetaryTotal"
type="MonetaryTotalType"/>
      <xs:element name="PaymentMeans"
type="PaymentMeansType"
                  minOccurs="0" maxOccurs="unbounded"/>
      <xs:element name="Note" type="xs:string"
                  minOccurs="0" maxOccurs="1"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
  ...
```

```
 ...
<xs:complexType name="MonetaryTotalType">
  <xs:sequence>
    <xs:element name="LineExtensionAmount"
type="AmountType"/>
    <xs:element name="TaxAmount" type="AmountType"/>
    <xs:element name="PayableAmount" type="AmountType"/>
  </xs:sequence>
</xs:complexType>

<xs:simpleType name="PaymentMethodCodeType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="30"/>  <!— credit transfer —>
    <xs:enumeration value="42"/>  <!— payment to bank
account —>
    <xs:enumeration value="48"/>  <!— bank card —>
    <xs:enumeration value="1"/>   <!— instrument not
defined —>
  </xs:restriction>
</xs:simpleType>
```

Three key definitions from the complete contract (remaining types on the next slide). The root `Invoice` element declares children in `xs:sequence` order — `PaymentMeans` is optional and repeatable, `Note` is optional. `MonetaryTotalType` enforces declaration order. `PaymentMethodCodeType` restricts values to four items — `"bank_transfer"` is not in the list.

> **Predict:** can `PayableAmount` appear before `TaxAmount`? Will `<PaymentMethodCode>42</PaymentMethodCode>` pass or fail?

Invoice XSD: Remaining Type Definitions

```
<xs:simpleType name="CurrencyCodeType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="SGD"/>
    <xs:enumeration value="USD"/>
    <xs:enumeration value="EUR"/>
    <xs:enumeration value="GBP"/>
  </xs:restriction>
</xs:simpleType>

<xs:complexType name="AmountType">
  <xs:simpleContent>
    <xs:extension base="xs:decimal">
      <xs:attribute name="currencyID"
type="CurrencyCodeType" use="required"/>
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>

<xs:complexType name="PartyType">
  <xs:sequence>
    <xs:element name="Name" type="xs:string"/>
    <xs:element name="Address" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
...
```

```
...
<xs:complexType name="InvoiceLineType">
  <xs:sequence>
    <xs:element name="ItemName" type="xs:string"/>
    <xs:element name="Quantity" type="xs:positiveInteger"/>
    <xs:element name="LineAmount" type="AmountType"/>
  </xs:sequence>
</xs:complexType>

<xs:complexType name="InvoiceLinesType">
  <xs:sequence>
    <xs:element name="InvoiceLine" type="InvoiceLineType"
                minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
```

`AmountType` combines a decimal value with a required `currencyID` attribute — referenced by `LineAmount`, `TaxAmount`, and `PayableAmount`. `InvoiceLineType` declares `Quantity` as `xs:positiveInteger` — the type that rejects `-3` in the invalid invoice. `PartyType` is shared by `Supplier` and `Customer`.

The Complete Invoice XSD: Answers

> **No** — `xs:sequence` enforces declaration order, so `PayableAmount` must come after `TaxAmount`. And **42 passes** — it is in the enumeration (payment to bank account).

The compositor determines ordering. The restriction determines allowed values. These are the two constraints that caused Failure 1's two rejections.

The Element Contract Card

The systematic way to "read the contract" — one card per element:

Field	Where to find it	Example (`InvoiceLine`)
Name	`xs:element name="…"`	`InvoiceLine`
Parent	Enclosing type containing this declaration	`InvoiceLines`
Compositor	`xs:sequence` / `xs:choice` / `xs:all`	`xs:sequence`
Cardinality	`minOccurs` / `maxOccurs` (defaults: 1/1)	(1, N)
Type	`type="…"` or inline definition	`InvoiceLineType` (complex)
Children	Child elements inside the compositor	ItemName, Quantity, LineAmount
Restrictions	`xs:restriction` facets (if simple type)	N/A (complex type)
Attributes	`xs:attribute` declarations	None

This is the formalised version of "read the contract" — the same skill from Topic 2, now applied to XSD element definitions instead of ERD (min, max) pairs.

Xpath: Navigating the Tree

XPath describes a walk through the tree: start from a context node, follow an **axis**, filter by **node test**, optionally filter by **predicate**.

XPath Expression	What It Does	Output
`/Invoice/Supplier/Name`	Child steps: Invoice → Supplier → Name	`Kopi Tech Pte Ltd`
`//InvoiceLine/LineAmount`	Descendant: any InvoiceLine, then child LineAmount	`12000.00`, `250.00`, `3000.00`
`//InvoiceLine[Quantity > 3]/ItemName`	Predicate filter: only lines with Quantity > 3	`SSL Certificate`, `Technical Support (Hours)`
`//InvoiceLine[1]/ItemName`	Position predicate: first InvoiceLine	`Cloud Hosting (Annual)`
`/Invoice/MonetaryTotal/PayableAmount/@currencyID`	Attribute axis: the `currencyID` attribute	`SGD`

Three concepts make all XPath expressions readable:

1. **Axis** — direction: `/` = child, `//` = descendant, `@` = attribute
2. **Node test** — filter by element or attribute name
3. **Predicate** — `[ … ]` additional filter: value, attribute, or position

Xpath: Predict the Results

On the invoice tree (3 line items: quantities 1, 5, 20):

> **Predict:** how many results does `/Invoice/InvoiceLines/InvoiceLine[Quantity > 3]/ItemName` return? What values?

Xpath: Results

> **Two results:** "SSL Certificate" (quantity 5) and "Technical Support (Hours)" (quantity 20). The predicate `[Quantity > 3]` filters out the first line item (quantity 1).

The XPath walks: child → child → child, filters by predicate, then takes one more child step to `ItemName`.

Why this matters after validation: against a validated document, `/Invoice/InvoiceLines/InvoiceLine/LineAmount` is a **guaranteed extraction** — the XSD requires the path to exist. Against an unvalidated document, an empty result could mean "no data" or "wrong path" or "wrong namespace" — you cannot tell which.

The Four-Check Model

The validator walks the XML tree top-down. At each element, four checks:

Check	What it asks	Failure 1 example
1. Name	Is this element declared in the XSD?	(no error — names are valid)
2. Compositor	Correct position relative to siblings?	`TaxAmount` before `LineExtensionAmount` — `xs:sequence` violated
3. Cardinality	Right number of occurrences?	Two `<Note>` elements — `maxOccurs="1"` violated
4. Content	Value satisfies type/restrictions?	`"bank_transfer"` not in enumeration; `-3` not a positive integer

After checking, the validator **recurses** into children and repeats the same four checks at each level.

```
Parse → Validate → Extract
  │         │          │
  │         │          └── XPath on validated tree (deterministic)
  │         └── Top-down walk: name → compositor → cardinality → content
  └── Build the tree (well-formedness)
```

Why Validation Belongs at the Boundary

This is Topic 5's constraint argument, applied to data crossing an **organisational boundary**.

Enforcement Concept	Topic 5 (Relational)	Topic 7 (XSD)
Single enforcement point	Database schema (DDL)	XSD + local validation
What it rejects	Invalid rows (wrong type, duplicate key)	Invalid documents (wrong element, wrong order, wrong type)
What it protects	All queries, all clients	All submissions, regardless of source
Where it runs	At write time (INSERT/UPDATE)	At the boundary (before submission)
Feedback latency	Milliseconds (same session)	Seconds locally; **24–48 hours** at the portal

Local validation collapses the feedback loop from days to seconds. The invoice that reaches the portal has already passed the same XSD. Rejection for structural reasons becomes impossible.

Well-Formed vs Valid: Now We Can Name the Check

Earlier we asked: is a well-formed document with `TaxAmount` before `LineExtensionAmount` valid?

Now we can name exactly which check catches it — the **compositor check**. The four-check model gives us precise vocabulary:

Earlier question	Answer with four-check vocabulary
Well-formed?	**Yes** — tags match, single root, proper nesting. The parser builds the tree.
Valid?	**No** — the **compositor check** fails. `xs:sequence` requires `LineExtensionAmount` before `TaxAmount`.

The distinction that seemed definitional earlier is now **mechanistic** — we know *which step in which model* catches the error.

Why XPath Extraction Is Reliable Only After Validation

`/Invoice/InvoiceLines/InvoiceLine/LineAmount` returns empty. Five possible reasons:

1. Element named `<lineAmount>` (case differs — XML is case-sensitive)
2. Amount stored as attribute, not child element
3. Element in a namespace — path needs a prefix
4. Element genuinely absent — sender omitted it
5. Document is malformed — parser could not build the tree

All five produce the **same symptom**: empty result. Without validation, you cannot distinguish them.

After validation: reasons 1, 2, 3, and 5 are eliminated. Only "no data" remains — a **business answer**, not a debugging problem.

XSD vs DTD: The Expressiveness Gap in Action

Same document, two validators, two results:

Invalid content	DTD Result	XSD Result	Why DTD Msses It
`<Quantity>-3</Quantity>`	Pass	**Fail** — `xs:positiveInteger`	DTD only knows "Quantity contains text"
`<PaymentMethodCode>bank_transfer</PaymentMethodCode>`	Pass	**Fail** — not in enumeration	DTD treats all content as `#PCDATA`

DTD can express structure and cardinality but treats all content as text. XSD closes the gap with **typed content** — the schema validates not just that an element exists, but that its value is correct.

This is why standards bodies (UBL, XBRL, HL7) use XSD, not DTD.

The Trust Boundary Distinction

Where does XML/XSD fit? Not against a workload — against a **trust boundary**.

Inside an organisation (high trust):

- One team controls writer and reader
- Schema changes weekly — coordinated in the same sprint
- Schema-on-read is a reasonable default (Topic 6's posture)

Between organisations (low trust):

- Writer and reader are different entities — different companies, codebases, release schedules
- The XSD is the only shared artefact
- Validation is the only defence against the other side's bugs

> **Predict:** a company has two data flows — (a) an internal microservice sends JSON events to an analytics pipeline (same team, schema changes weekly); (b) a monthly XML tax filing to a revenue authority (published XSD, rejections on failure). What enforcement position for each?

Trust Boundary: Answer

Flow (a): schema-on-read (Topic 6's posture) — internal, high trust, rapidly evolving. Validation overhead would slow iteration without adding safety. **Flow (b):** validator-rejects (this topic's posture) — crosses an organisational boundary to a regulator. The XSD is the only shared contract. Errors cost resubmission delays.

Cost of choosing wrong: applying XSD to flow (a) creates overhead without safety. Omitting validation from flow (b) produces stuck submissions — exactly Failure 1.

When XML/XSD Is the Wrong Choice

XML validation adds value at trust boundaries. But:

- **High-trust, rapidly evolving APIs:** XSD overhead slows iteration without adding safety — use JSON with schema-on-read (Topic 6)
- **Semantic correctness:** an invoice can pass every XSD check (correct types, ordering, cardinalities) but contain a **wrong total** — the schema guarantees structure, not truth
- **Cross-source data integration:** when the dominant need is identity alignment and schema-late evolution, Topic 8's RDF model serves better

Cost: XML/XSD trades flexibility for contract rigidity. That trade-off is appropriate at trust boundaries, costly elsewhere.

JSON / MongoDB (Topic 6) vs XML / XSD (Topic 7)

Dimension	JSON / MongoDB	XML / XSD
Tree structure	Objects + arrays	Elements + child elements
Path navigation	Dot notation (`lineItems.sku`)	XPath (`/Invoice//InvoiceLine/LineAmount`)
Nested filtering	`$elemMatch` (same-element predicate)	XPath predicate (`InvoiceLine[Quantity > 3]`)
Schema posture	Schema-on-read (default)	Contract-first: validate before submission
Structural variation	Natural — different fields per document	Constrained — must be declared in XSD
Cardinality constraints	Bounded-growth rules (application-enforced)	`minOccurs`/`maxOccurs` (schema-enforced)
Primary context	Application-internal (one team, high trust)	Cross-boundary exchange (regulated, low trust)
Who controls the schema?	The application team	The standard body or regulator
Enforcement spectrum	"Nothing rejects"	"Validator rejects"

The Validation Procedure: Seven Steps

Step	Action	Tool
1. Parse	Check well-formedness	`xmllint file.xml`
2. Read the contract	Fill in element contract cards — root → recurse	XSD (human reading)
3. Predict	Walk document top-down, apply four checks mentally	Contract cards
4. Validate (XSD)	Compare actual errors to predictions	`xmllint --schema schema.xsd file.xml`
5. DTD check (if exists)	Note what DTD catches vs misses	`xmllint --dtd file.dtd file.xml`
6. Extract	XPath on validated document — predict first, then verify	`xmllint --xpath 'expr' file.xml`
7. Close the loop	Invalid → rejected → fixed → passes → extract works	Full chain

The same procedure applies to JSON with JSON Schema as the contract.

JSON Schema: The Same Discipline for Modern Systems

Failure 3's payment API had documentation but no contract. **JSON Schema** provides the same validation capability as XSD — for JSON data:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "required": ["amount", "currency", "paymentDate", "merchantId"],
  "properties": {
    "amount":      { "type": "number", "exclusiveMinimum": 0 },
    "currency":    { "type": "string", "enum": ["SGD", "USD", "EUR", "GBP"] },
    "paymentDate": { "type": "string", "format": "date" },
    "merchantId":  { "type": "string", "pattern": "^M-\\d{4}$" }
  },
  "additionalProperties": false
}
```

Validating Failure 3's payload against this schema produces five errors:

```
  Path: ['amount']     |  Error: '150.00' is not of type 'number'
  Path: ['currency']   |  Error: 'sgd' is not valid under any of the given schemas
  Path: (root)         |  Error: 'paymentDate' is a required property
  Path: (root)         |  Error: 'merchantId' is a required property
  Path: (root)         |  Error: Additional properties are not allowed
                              ('payment_date', 'merchant_id' were unexpected)
Total errors: 5
```

JSON Schema: The Four-Check Model Applies

XSD Check	JSON Schema Equivalent	Failure 3 Example
Name (undeclared element)	`additionalProperties: false`	`payment_date` → rejected (expected `paymentDate`)
Compositor (ordering)	N/A — JSON objects are unordered	—
Cardinality (`minOccurs`)	`required: [ ... ]`	Missing required fields detected
Content (type/restriction)	`type`, `enum`, `pattern`	`"150.00"` (string) vs `number`; `"sgd"` vs `"SGD"` enum

The last row reveals the key structural difference: XML is **ordered** (sibling elements have a defined sequence); JSON objects are **unordered**. XSD can enforce ordering; JSON Schema cannot.

Predict: the partner sends `{ "amount": 150, "currency": "SGD", "paymentDate": "2025-02-13", "merchantId": "M-2001", "notes": "urgent" }`. Pass or fail?

JSON Schema: additionalProperties

> **Fail.** The payload has an undeclared field `"notes"`, and `"additionalProperties": false` rejects any property not listed in `"properties"`.

The data types and values are all correct. But the schema's strictness catches the extra field.

Connection to Topic 6: MongoDB's `$jsonSchema` validator uses this same JSON Schema vocabulary. The difference is posture: in Topic 6, `$jsonSchema` was **opt-in** validation added to a permissive system. Here, JSON Schema is the **contract-first** posture — the schema defines structure before data exists.

Cost: `additionalProperties: false` rejects legitimate extensions from evolving partners — every schema change requires coordination across organisations. The same rigidity trade-off as XSD, now in JSON notation.

XSD → JSON Schema Mapping

Validation Concept	XSD	JSON Schema
Required fields	`minOccurs="1"` (default)	`"required": ["field1", "field2"]`
Optional fields	`minOccurs="0"`	Absent from `required` array
Data type	`xs:integer`, `xs:date`, `xs:decimal`	`"type": "integer"`, `"format": "date"`, `"type": "number"`
Allowed values	`xs:enumeration`	`"enum": [ ... ]`
Value patterns	`xs:pattern`	`"pattern": "^ ... "`
Numeric ranges	`xs:minInclusive` / `xs:maxInclusive`	`"minimum"` / `"maximum"`
Array cardinality	`minOccurs` / `maxOccurs` on repeated element	`"minItems"` / `"maxItems"`
Ordering	`xs:sequence` enforces order	No ordering constraint
Schema format	XML	JSON

Worked Example: Tracing the Valid Invoice

Step 1 — Parse. Matched tags, single root, proper nesting. Well-formed.

Step 2 — Read the contract. Three key contract cards:

Field	`Invoice` **(Root)**	`InvoiceLine`	`PaymentMethodCode`
Parent	(root)	`InvoiceLines`	`PaymentMeans`
Compositor	`xs:sequence`	`xs:sequence`	— (simple type)
Cardinality	(1,1)	(1, N)	(1,1)
Type	complex	complex	`PaymentMethodCodeType`
Children	InvoiceNumber, IssueDate, Supplier, Customer, InvoiceLines, MonetaryTotal, PaymentMeans (0,N), Note (0,1)	ItemName, Quantity, LineAmount	— (leaf)
Restrictions	—	—	enum: "30", "42", "48", "1"

Worked Example: Predict, Validate, Extract

Step 3 — Predict. Elements in sequence order? Yes. All required present? Yes. Quantities: 1, 5, 20 — all positive integers. PaymentMethodCode: "30" — in enumeration. Prediction: **zero errors**.

Step 4 — Validate. `xmllint --schema invoice_schema.xsd invoice_valid.xml` → `✓ Valid`. Matches prediction.

Step 6 — Extract. XPath on the validated document — predict each result, then verify:

```
$ xmllint --xpath "/Invoice/Supplier/Name" invoice_valid.xml
→ "Kopi Tech Pte Ltd"                                        ✓ predicted

$ xmllint --xpath "//InvoiceLine[Quantity > 3]/ItemName" invoice_valid.xml
→ "SSL Certificate"
→ "Technical Support (Hours)"                                 ✓ 2 results predicted

$ xmllint --xpath "/Invoice/MonetaryTotal/PayableAmount/@currencyID" invoice_valid.xml
→ "SGD"                                                       ✓ predicted

$ xmllint --xpath "//InvoiceLine[1]/ItemName" invoice_valid.xml
→ "Cloud Hosting (Annual)"                                    ✓ predicted
```

The chain holds: **parse → validate → extract**.

The Invalid Invoice: Four Deliberate Errors

Error 1 — Compositor: `MonetaryTotal` and `InvoiceLines` swapped.

```
<Customer> ... </Customer>
<MonetaryTotal>                          <!— should appear AFTER InvoiceLines —>
  <LineExtensionAmount currencyID="SGD">500.00</LineExtensionAmount>
  <TaxAmount currencyID="SGD">45.00</TaxAmount>
  <PayableAmount currencyID="SGD">545.00</PayableAmount>
</MonetaryTotal>
<InvoiceLines>                           <!— should appear BEFORE MonetaryTotal —>
  <InvoiceLine> ... </InvoiceLine>
</InvoiceLines>
```

Error 2 — Content/type: `Quantity` is `xs:positiveInteger`.

```
    <Quantity>-3</Quantity>              <!— xs:positiveInteger rejects negative —>
```

Error 3 — Content/enumeration: `PaymentMethodCodeType` allows `"30"`, `"42"`, `"48"`, `"1"`.

```
  <PaymentMethodCode>bank_transfer</PaymentMethodCode>   <!— not in enumeration —>
```

Error 4 — Cardinality: `Note` has `maxOccurs="1"`.

```
<Note>Urgent delivery required.</Note>
<Note>Please confirm receipt.</Note>     <!— maxOccurs="1" allows only one —>
```

Worked Example: The Invalid Invoice

The four-check model on `invoice_invalid.xml`:

Check	What the validator finds	XSD rule violated
Name	No errors — all element names are valid	—
Compositor	`MonetaryTotal` before `InvoiceLines`	`xs:sequence` order violated
Cardinality	Two `<Note>` elements	`maxOccurs="1"` allows only one
Content	`Quantity` is `-3`; `PaymentMethodCode` is `"bank_transfer"`	`xs:positiveInteger` rejects negative; enumeration rejects non-numeric

`xmllint --schema invoice_schema.xsd invoice_invalid.xml` produces:

```
invoice_invalid.xml:14: element MonetaryTotal: Schemas validity error:
  Element 'MonetaryTotal': This element is not expected.
  Expected is ( InvoiceLines ).
invoice_invalid.xml:23: element Quantity: Schemas validity error:
  Element 'Quantity': '-3' is not a valid value of the atomic type
  'xs:positiveInteger'.
invoice_invalid.xml:29: element PaymentMethodCode: Schemas validity error:
  Element 'PaymentMethodCode': 'bank_transfer' is not a valid value of
  the atomic type 'PaymentMethodCodeType'.
invoice_invalid.xml:34: element Note: Schemas validity error:
  Element 'Note': This element is not expected.
```

Watch for error cascading: the compositor violation may cause additional positional errors for elements displaced by the earlier violation.

Check 1: Hospital Lab System

A hospital laboratory receives test orders from external clinics via XML. The XSD requires `<LabOrder>` to contain `<PatientID>`, `<TestCode>`, and `<Priority>` in `xs:sequence`, with `Priority` restricted to an enumeration: `"routine"`, `"urgent"`, `"stat"`.

A clinic submits:

```
<LabOrder>
  <TestCode>CBC-001</TestCode>
  <PatientID>P-4821</PatientID>
  <Priority>ASAP</Priority>
</LabOrder>
```

Which checks fail? How many distinct errors?

> **Two errors.** (1) Compositor: `TestCode` before `PatientID` violates `xs:sequence`. (2) Content: `"ASAP"` is not in the enumeration. The name check passes — all element names are declared. Cardinality: each appears once — correct.

Check 2: Music Streaming Royalty Report

A music streaming platform submits quarterly royalty reports to rights holders via XML. The XSD requires `<RoyaltyReport>` to contain `<ReportPeriod>` (exactly once), `<ArtistPayment>` (`minOccurs="1"`, `maxOccurs="unbounded"`), and `<TotalAmount>` (exactly once, `xs:decimal`).

A report arrives with `<TotalAmount>TBD</TotalAmount>` and zero `<ArtistPayment>` elements.

Which checks fail?

> **Two errors.** (1) Cardinality: zero `<ArtistPayment>` elements violates `minOccurs="1"`. (2) Content: `"TBD"` is not a valid `xs:decimal`. The compositor check passes (if elements present are in order). This is the same cardinality concept from Topic 2 — (min, max) applied to tree children.

Check 3: Tax Authority: DTD vs XSD

A tax authority accepts filings validated by DTD only (legacy system). A company submits `<TaxAmount>-500.00</TaxAmount>` (negative — a data entry error) and `<FilingCode>annual_return</FilingCode>` (the code list requires `"AR-01"`).

The DTD validation passes. Why?

> **DTD treats all content as text.** `-500.00` is text — DTD cannot check numeric types. `"annual_return"` is text — DTD cannot check enumerations on element content. Both errors would be caught by XSD: `xs:decimal` with `xs:minInclusive value="0"` rejects the negative, and `xs:enumeration` rejects the unlisted code. This is the same pattern from the DTD vs XSD comparison — the same document, two validators, two results.

Check 4: Pharmaceutical Supply Chain

A pharmaceutical company exchanges shipment records with a distributor. The JSON API contract (JSON Schema) requires:

```
{ "required": ["batchId", "quantity", "expiryDate"],
  "properties": {
    "batchId": { "type": "string", "pattern": "^B-\\d{6}$" },
    "quantity": { "type": "integer", "minimum": 1 },
    "expiryDate": { "type": "string", "format": "date" }
  },
  "additionalProperties": false }
```

The distributor sends `{ "batch_id": "B-123", "quantity": 0, "expiryDate": "2025-06-15" }`. How many errors?

> **Three errors.** (1) `"batch_id"` is undeclared — `additionalProperties: false` rejects it. (2) `"batchId"` is missing — `required` check fails. (3) `"quantity": 0` fails — `minimum: 1`. The pattern check on `batchId` doesn't fire because the field is absent. Same discipline, JSON notation.

Check 5: Contract-Card Reading (XSD Without XML)

Given the `MonetaryTotalType` from Slide 19 — `xs:sequence` with children `LineExtensionAmount`, `TaxAmount`, `PayableAmount`, none marked `minOccurs="0"`:

(a) How many of the three child elements are required? (b) Can `PayableAmount` appear before `TaxAmount`? (c) Can `LineExtensionAmount` appear twice?

> **(a) All three required** — no `minOccurs="0"` on any, so defaults apply: `minOccurs="1"`. **(b) No** — `xs:sequence` enforces declaration order. **(c) No** — `maxOccurs` defaults to 1. You answer from the XSD alone — no XML document, no validator. This is the contract-card method from Slide 22: read the schema, predict the outcome. The skill that took the developer in Failure 2 thirty to ninety minutes per field becomes a two-minute exercise.

Check 6: Cross-Topic: The Enforcement Spectrum

Three topics, three positions:

- **Topic 5** (engine rejects): the SQL engine blocks an invalid INSERT at write time — invalid data is *impossible*.
- **Topic 6** (nothing rejects): MongoDB accepts any document shape by default — invalid data is *undetectable* unless `$jsonSchema` is added.
- **Topic 7** (validator rejects): XSD or JSON Schema checks a document *after* creation but *before* system entry — invalid data is *detectable and stoppable* at the boundary.

Which position is appropriate when?

> **Engine rejects** when you control the database and want maximum safety. **Validator rejects** when data crosses a trust boundary and you need to check what someone else produced. **Nothing rejects** when one team controls both writer and reader and values flexibility over enforcement. The position is a design choice driven by the trust boundary, not a quality ranking.

The Enforcement Spectrum: Three Positions Filled

```
Enforcement Spectrum
═══════════════════════════════════════════════════════════════
  Engine rejects          Validator rejects         Nothing rejects
  (Topic 5)               (Topic 7)                 (Topic 6)
  ─────────               ─────────────             ─────────────────

  DDL constraints         XSD / JSON Schema         Schema-on-read
  GRANT / REVOKE          at the boundary           (MongoDB default)
  Transactions            before submission

  ◄── Strictest                                Most flexible ──►

  You control the DB      Data crosses a            One team controls
  and want maximum        trust boundary            writer and reader;
  safety                                            flexibility > safety
═══════════════════════════════════════════════════════════════
```

Three positions, three different trust assumptions. Not a quality ranking — a **design choice** driven by who you trust.

Concepts Planted Here

Concept	Introduced As	Recurs In
Validator rejects	XSD / JSON Schema at boundary	Topic 8 (contrast: ontology does not reject), Topic 11 (boundary validation)
Element contract card	Template for reading XSD	Topic 8 (property card for RDF ontology)
(min, max) in tree form	`minOccurs` / `maxOccurs`	Topic 11 (cardinality across all five models)
Trust boundary	Organisational line where validation is the only defence	Topic 11 (data-flow boundaries between models)
Parse → Validate → Extract	Three-step reliability chain	Topic 11 (validation pipeline at model boundaries)
Contract-first posture	Schema defines structure before data exists	Topic 8 (contrast: open-world assumption), Topic 11

Course Map

```
Part 1 — Relational Foundation
├── Ch 1  Introduction to Data Models
├── Ch 2  Conceptual Modelling (ERD)
├── Ch 3  Normalisation (BCNF & Star Schemas)
├── Ch 4  SQL Querying (FWGHOS Pipeline)
└── Ch 5  Enforcement (Constraints, Access Control, Transactions, Indexes)

Part 2 — Alternative Models
├── Ch 6  Document Model (JSON & MongoDB)
├── Ch 7  XML for Contracts (XSD & XPath)  ◄── you are here
├── Ch 8  RDF and Knowledge Graphs
├── Ch 9  SPARQL Querying
└── Ch 10 Property Graphs (Cypher)

Part 3 — Capstone
└── Ch 11 Multi-Model Design
```

What carries forward: the **element contract card** becomes the property card in Topic 8 (same skill, RDF ontology). The **trust boundary** becomes the data-flow boundary in Topic 11. The **(min, max)** pairs and the **parse → validate → extract** chain recur across all five models.

Topic 8 introduces RDF — where the ontology *describes* structure but does not *enforce* it. A direct contrast: XSD is **prescriptive** (reject violations); RDF ontology is **descriptive** (describe structure, accept everything).

Lab Preview

Topic 7 Lab — XML for Contracts: Schema Validation (XSD) and XPath Extraction

Part	What you do
Part 1	Read the XSD — fill in five element contract cards
Part 2	Predict validation failures using the four-check model, then validate
Part 3	Compare DTD vs XSD — same document, two validators
Part 4	XPath extraction — predict results, then verify
Part 5	JSON Schema validation — Failure 3's payload
Part 6	Cross-model comparison table (Topics 5–7)

Format: reasoning + code execution (Google Colab with `%%xpath` magic)

File: `07-lab-xml-contracts.ipynb`